# IERG4320/IEMS5726: Data Science in Practice

# Lecture 5: Data Representation III

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

香港中文大學
The Chinese University of Hong Kong

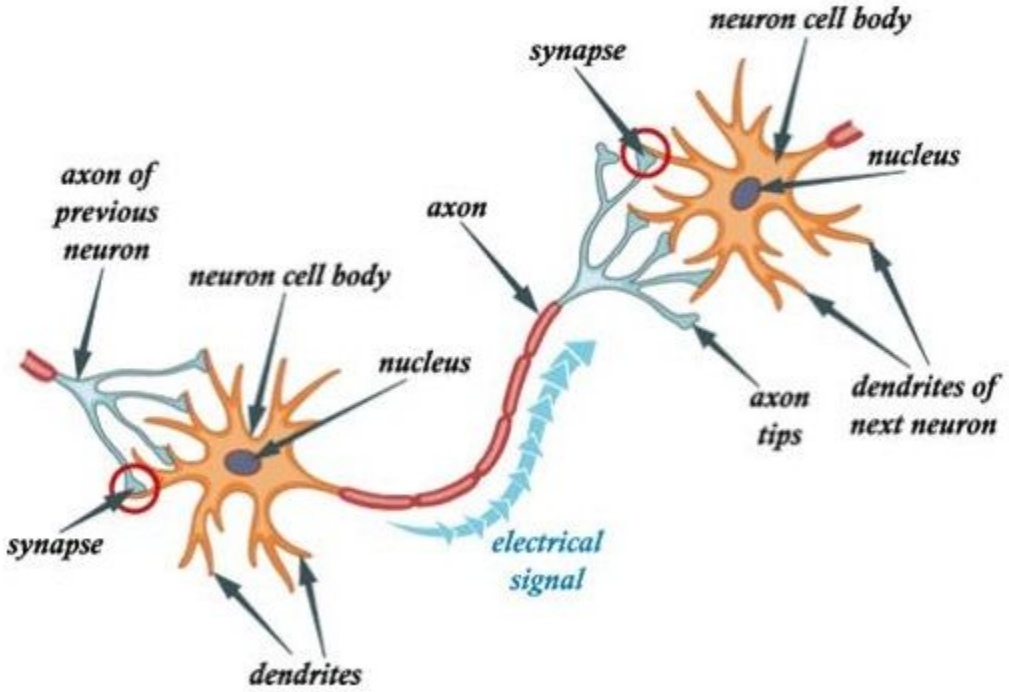
# Outline

1. Introduction to Artificial Neural Network
2. Data Representation by Autoencoder
3. Google Colaboratory
4. Data Representation by Transformer

All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

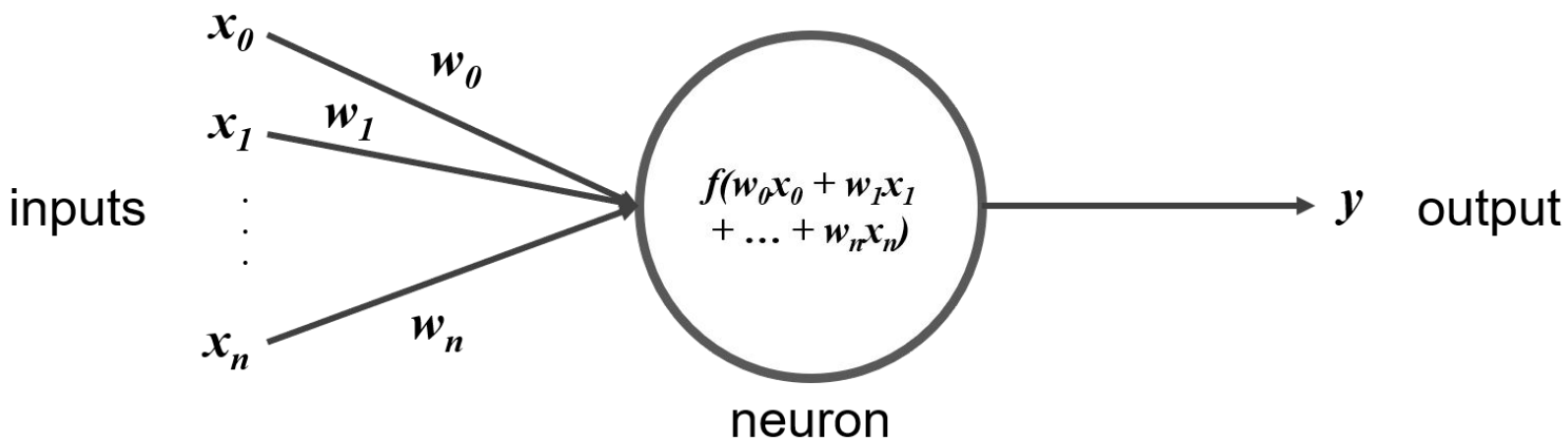
# Neural Network (NN) / Artificial Neural Network (ANN)

- A computational model inspired by the biological neural networks in the human body
- A model involving a set of **neurons** connected to one another in some specific ways
- Can be used to model **non-linear** relations between inputs and outputs



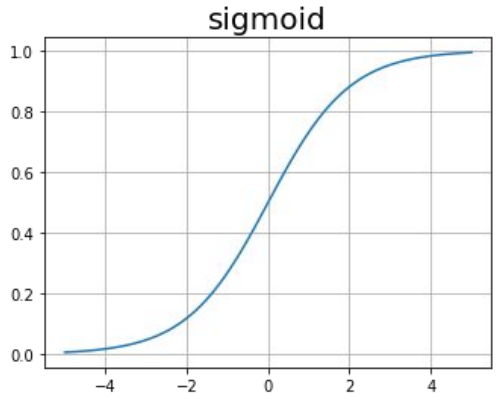
# Basic Unit: Neurons

- A **neuron** is the basic unit in a neural network (also called a **node**)
- Each input is associated with a **weight**
- A neuron applies a function f on the **weighted sum** of the inputs, and generates its **output**

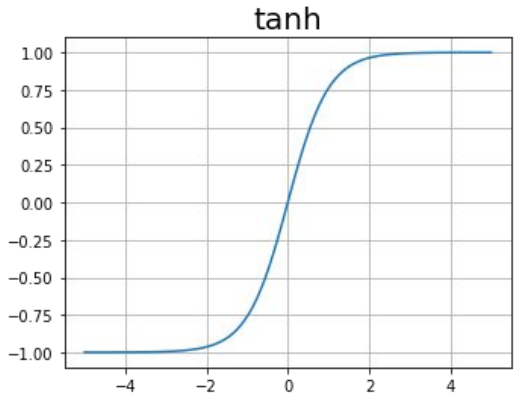


# Activation Functions

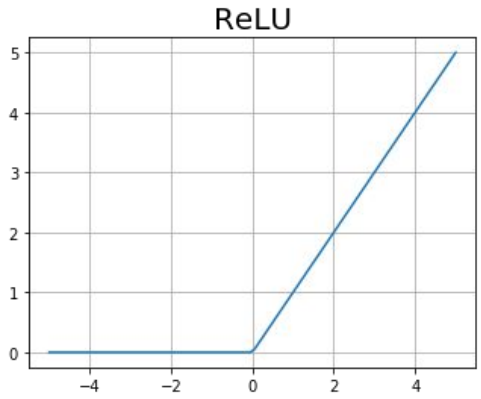
- f is called the **activation function** of the neuron
- It is usually a **non-linear** function
- Using non-linear activation functions allow neural networks to model **non-linear** relations




$$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$



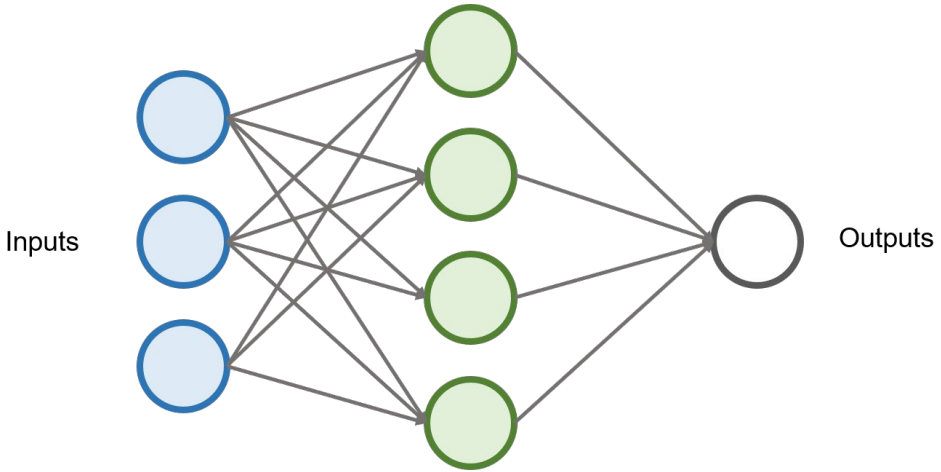

$$\tanh(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$$




$$\text{ReLU}(x) = \max(0, x)$$

# Feed-forward Neural Networks

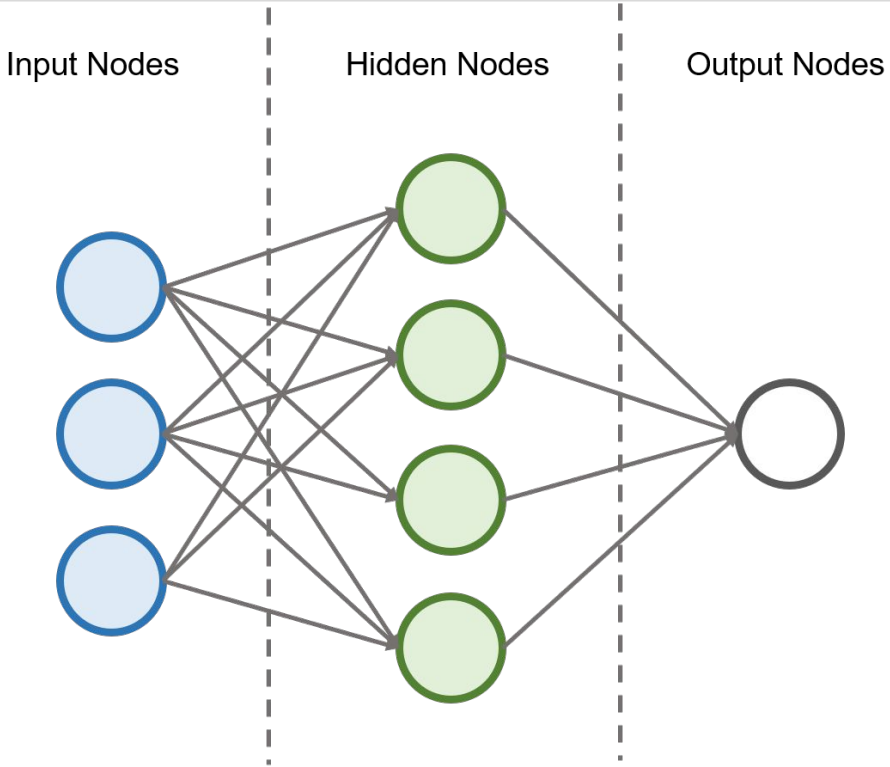
- In a **feed-forward** neural network, computation is performed in a **single direction**
- Neurons are arranged in one or more **layers**
- Nodes are connected by **edges**, with **weights** associated with the edges



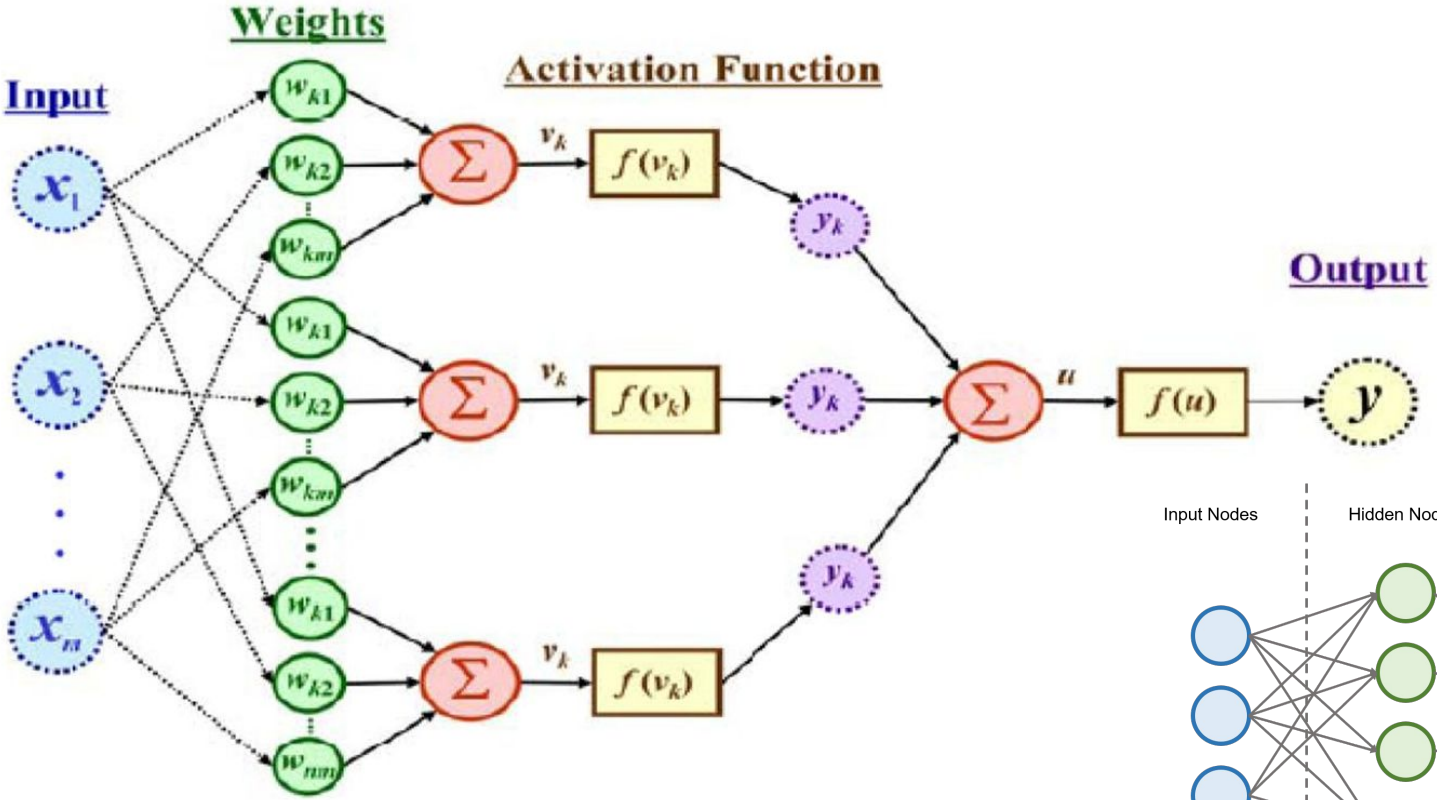
# Feed-forward Neural Networks

- A feed-forward neural network has **three different types of nodes**:

1. **Input Nodes**: Pass inputs to the neural network, no computation is performed
2. **Hidden Nodes**: Perform computations and transfer information from the previous layer to the next layer
3. **Output Nodes**: Represents the output of the neural network



# More Detailed Visualization on a Neuron Network



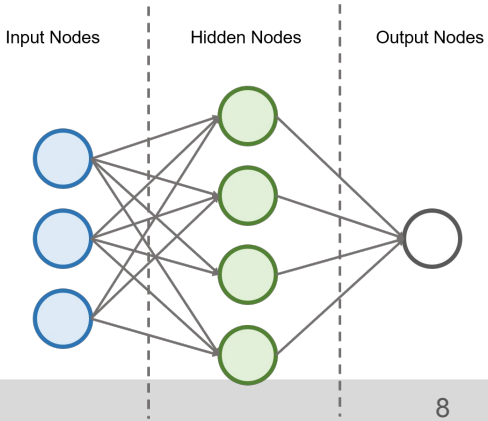




Image credit: Prof. Ahmad Aljebaly, Western Michigan University

# Multi-Layer Perceptron (MLP)

- A neural network that is consist of **one or more** hidden layers are also called a **multi-layer perceptron (MLP)**
- A multi-layer perceptron can be used to learn **non-linear functions**
- Also called a **fully connected** neural network (each node in one layer is connected to all nodes in the next layer)

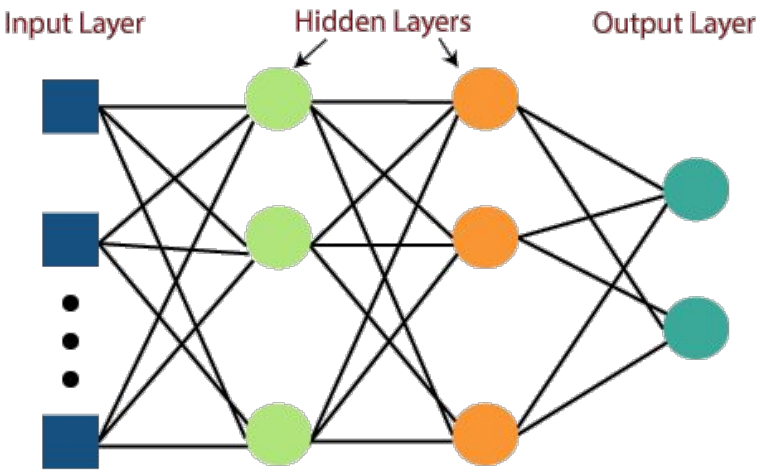



Image credit:
https://github.com/Thanasis1101/MLP-from-scratch

# Loss Function in Neural Networks

- A neural network can be easily designed to perform **multi-class classification**
- This is by creating **N output nodes** in the output layer
- A commonly used **loss function** in training neural networks for classification is the **logarithmic loss function (Log Loss)**
- Reference: http://scikit-learn.org/stable/modules/model_evaluation.html#log-loss

$$\text{logloss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log(p_{ic})$$

- N is number of training samples, C is the number of classes (labels)
- $y_{ic}$=1 if sample i belongs to class c, otherwise it is 0
- $p_{ic}$ is the **predicted probability** that sample i belongs to class c

# Parameters in a Neural Network

1. Number of layers.
2. Number of nodes of each layers.
3. Activation functions of each layer.
4. Optimizer: To change the learning rate and weights of neurons in the neural network to reach the minimum loss function
5. Learning rate: To control the step size for a model to reach the minimum loss function
6. Batch size: The number of training data sub-samples for the input
7. Epoch: The number of times a whole dataset is passed through the neural network model
8. Weights assigned to each neuron.
9. Bias assigned to each neuron.

# Steps in Training a Neural Network

1. Pick a neural network architecture.
2. Random Initialization of Weights and Bias.
3. Implementation of forward propagation algorithm to calculate hypothesis function for a set on input vector for any of the hidden layer.
4. Implementation of cost function for optimizing parameter values. One may recall that cost function would help determine how well the neural network fits the training data.
5. Implementation of back propagation algorithm to compute the error vector related with each of the nodes.
6. Use gradient checking method to compare the gradient calculated using partial derivatives of cost function using back propagation and using numerical estimate of cost function gradient.
7. Use gradient descent or advanced optimization technique with back propagation to try and minimize the cost function as a function of parameters or weights.

# Simple ANN: Initialize ANN

```python
import torch
import torch.nn as nn
# input size, hidden unit, output size, batch size, and the learning rate.
n_input, n_hidden, n_out, batch_size, learning_rate = 10, 15, 1, 100, 0.01
# randomly initialize the dummy input and the output target data (or tensor)
data_x = torch.randn(batch_size, n_input)
data_y = (torch.rand(size=(batch_size, 1)) < 0.5).float()
# define ANN model
model = nn.Sequential(nn.Linear(n_input, n_hidden),nn.ReLU(),
                      nn.Linear(n_hidden, n_out),nn.Sigmoid())
print(model)
loss_function = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

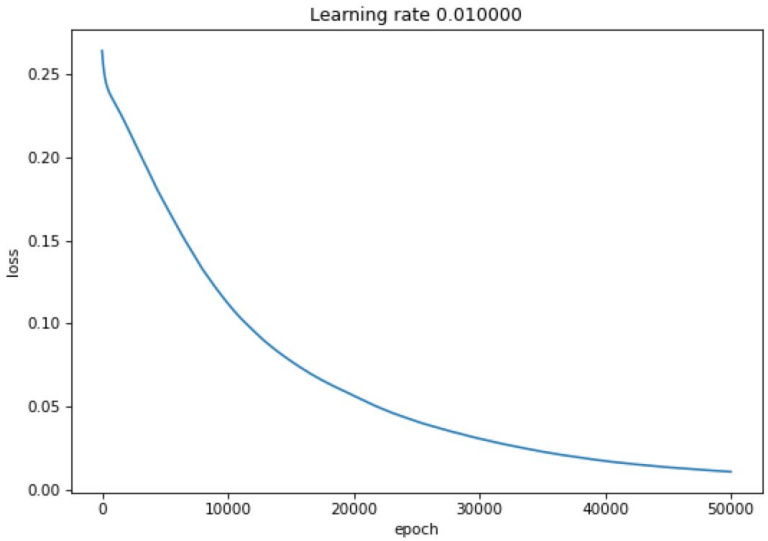
```
Sequential(
  (0): Linear(in_features=10, out_features=15, bias=True)
  (1): ReLU()
  (2): Linear(in_features=15, out_features=1, bias=True)
  (3): Sigmoid()
)
```

Reference:
https://towardsdatascience.com/build-a-simple-neural-network-using-pytorch-38c55158028d

# Simple ANN: Train the Model, Visualize Training Performance

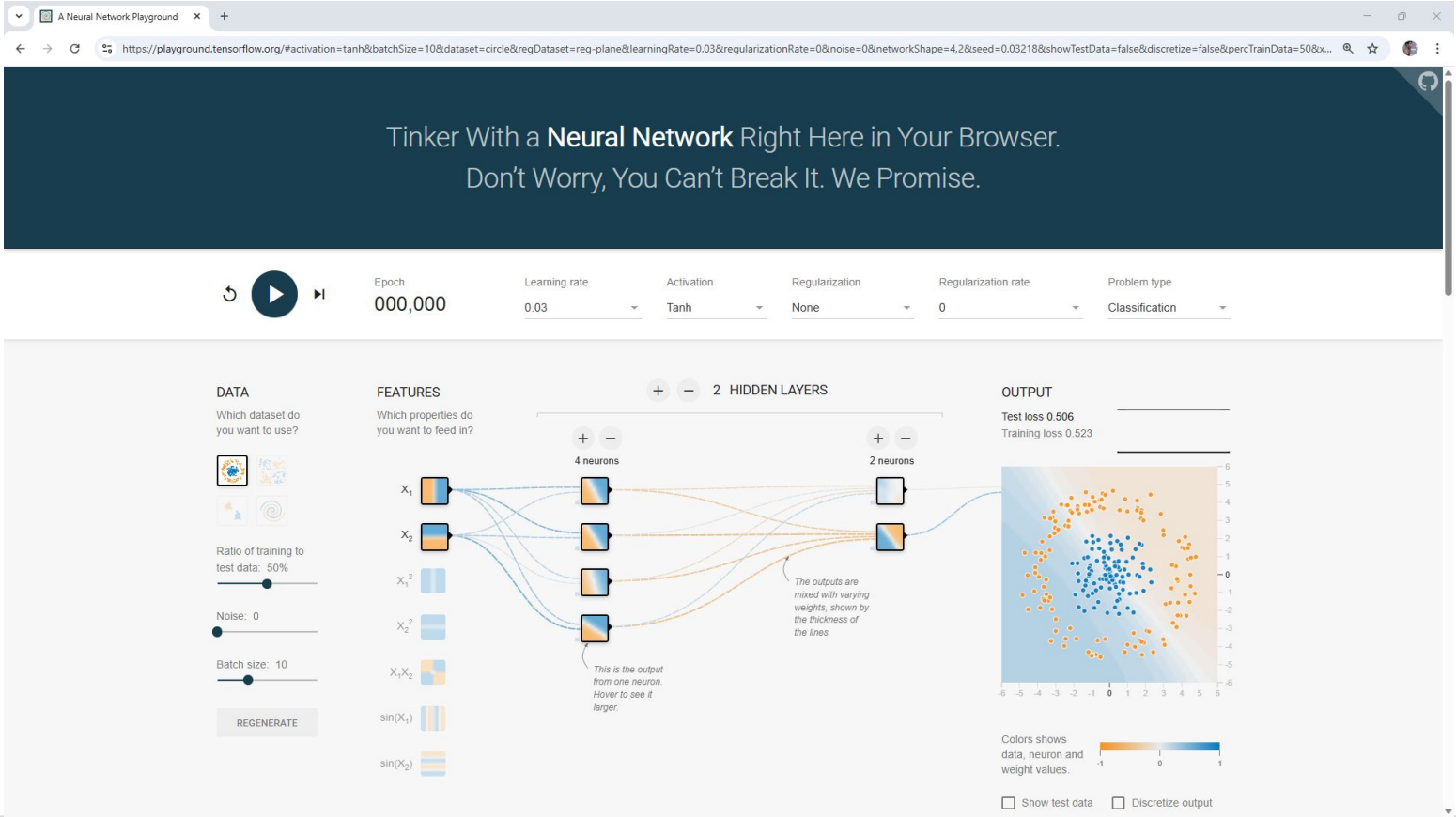
```
# Training Loop
losses = []
for epoch in range(50000):
    pred_y = model(data_x)  # Forward propagation
    loss = loss_function(pred_y, data_y)
    losses.append(loss.item())
    model.zero_grad()
    loss.backward()         # Backward propagation
    optimizer.step()        # Gradient descent

import matplotlib.pyplot as plt
plt.plot(losses)
plt.ylabel('loss')
plt.xlabel('epoch')
plt.title("Learning rate %f" % (learning_rate))
plt.show()
```




```
[<matplotlib.lines.Line2D object at 0x0000013226A89AD0>]
Text(0, 0.5, 'loss')
Text(0.5, 0, 'epoch')
Text(0.5, 1.0, 'Learning rate 0.010000')
```

# Tensorflow Playground: Visualizing Neural Network

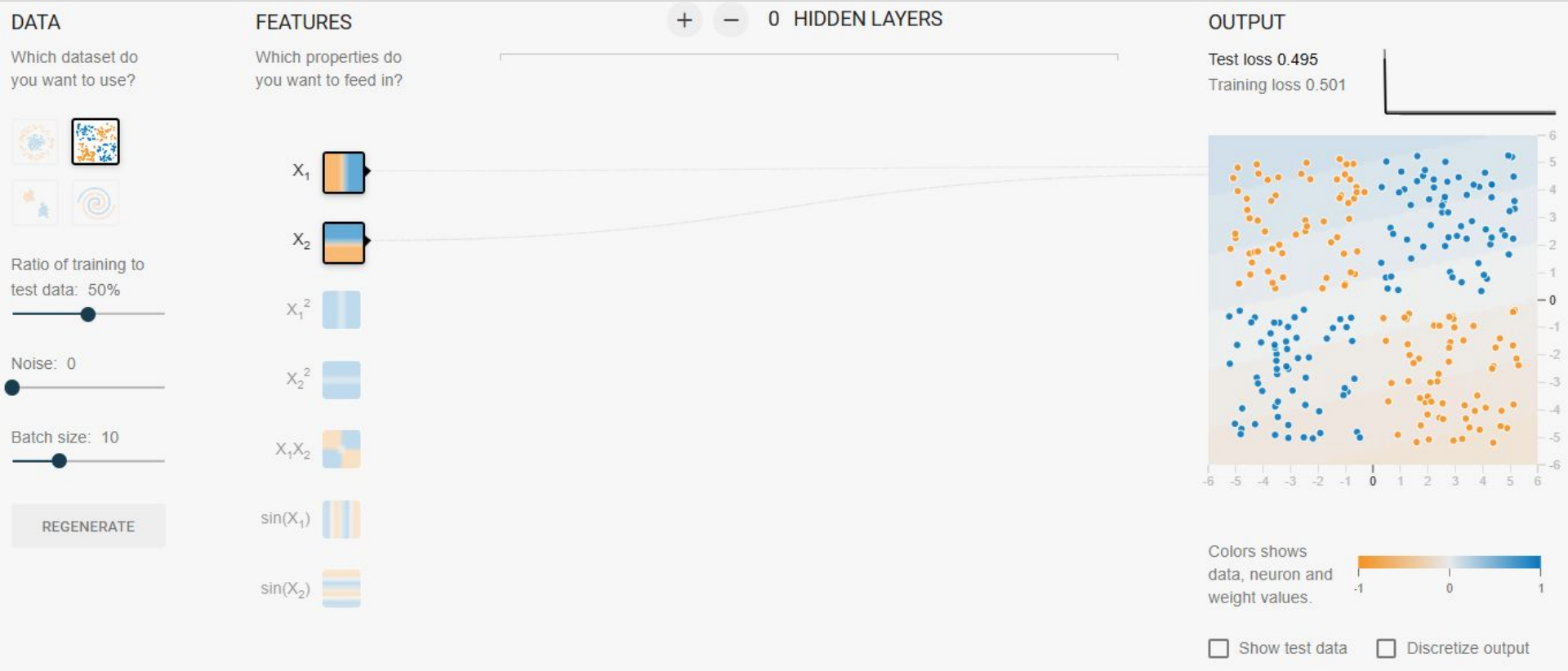




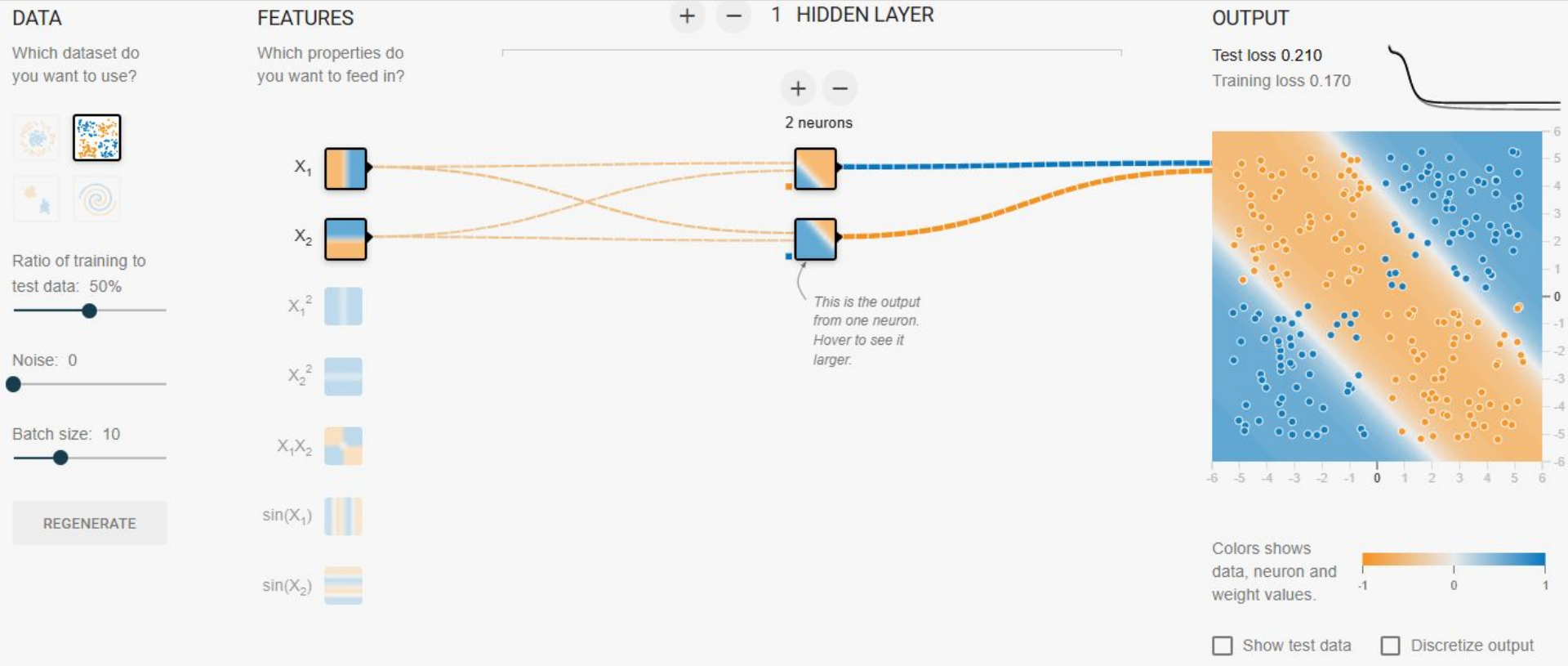

https://playground.tensorflow.org/

# Test #1: Two clusters with vertical and horizontal lines

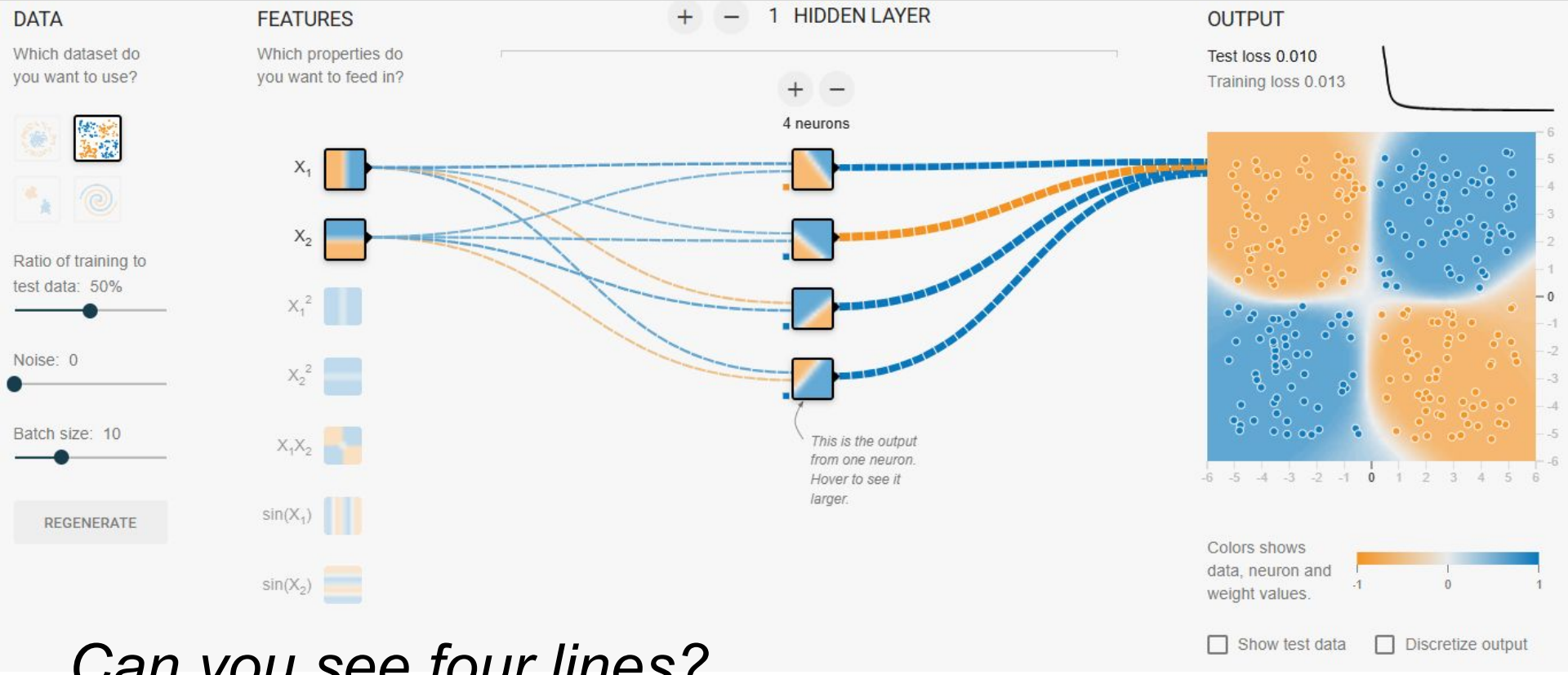
# Test #2: Non-linear clusters



# Test #2: Non-linear clusters - one more layer

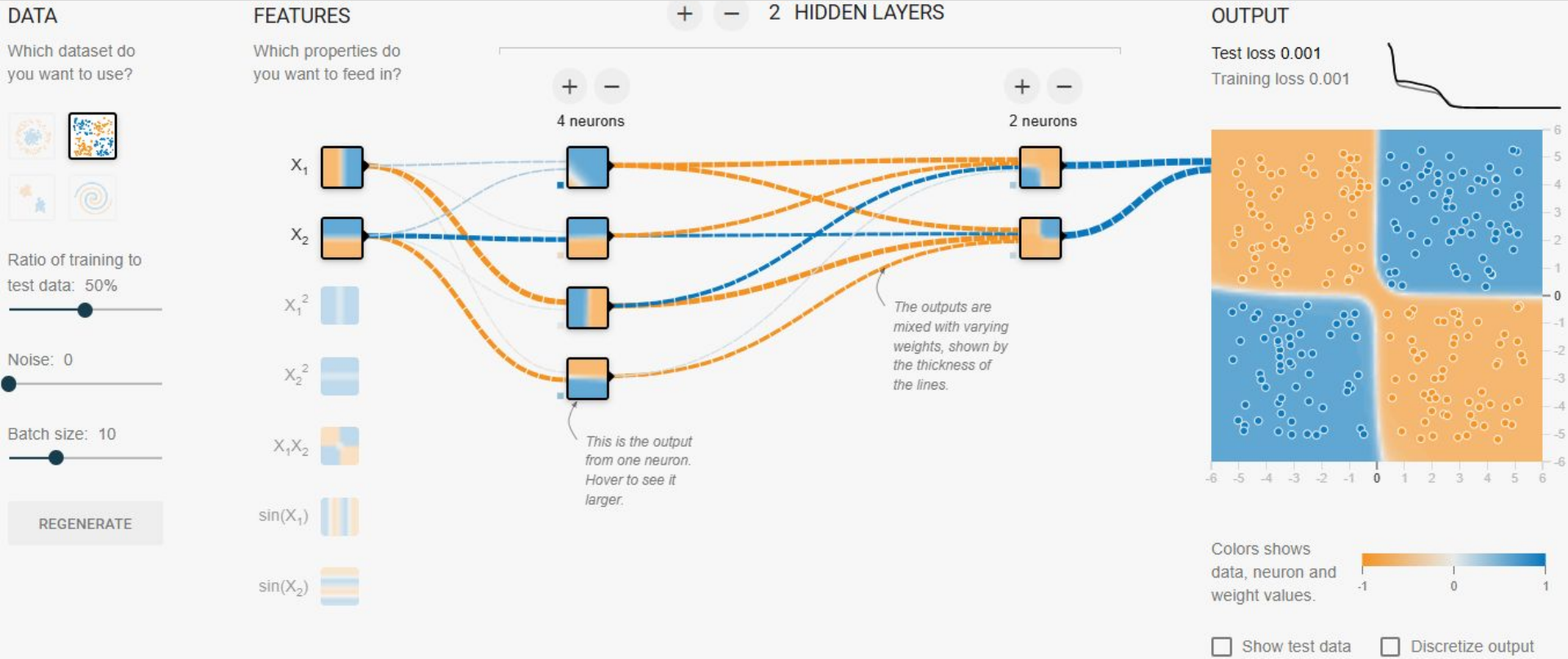


# Test #2: Non-linear clusters - more neurons

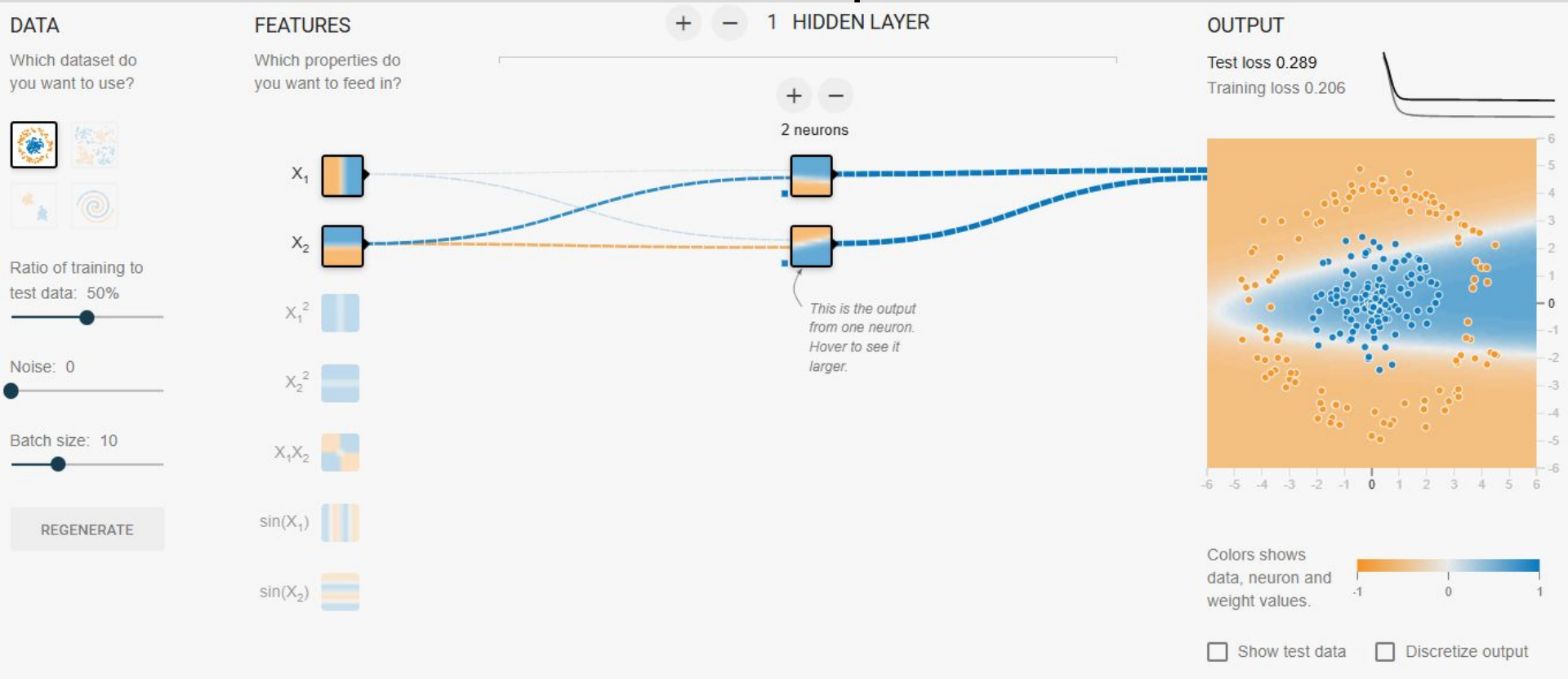



*Can you see four lines?*

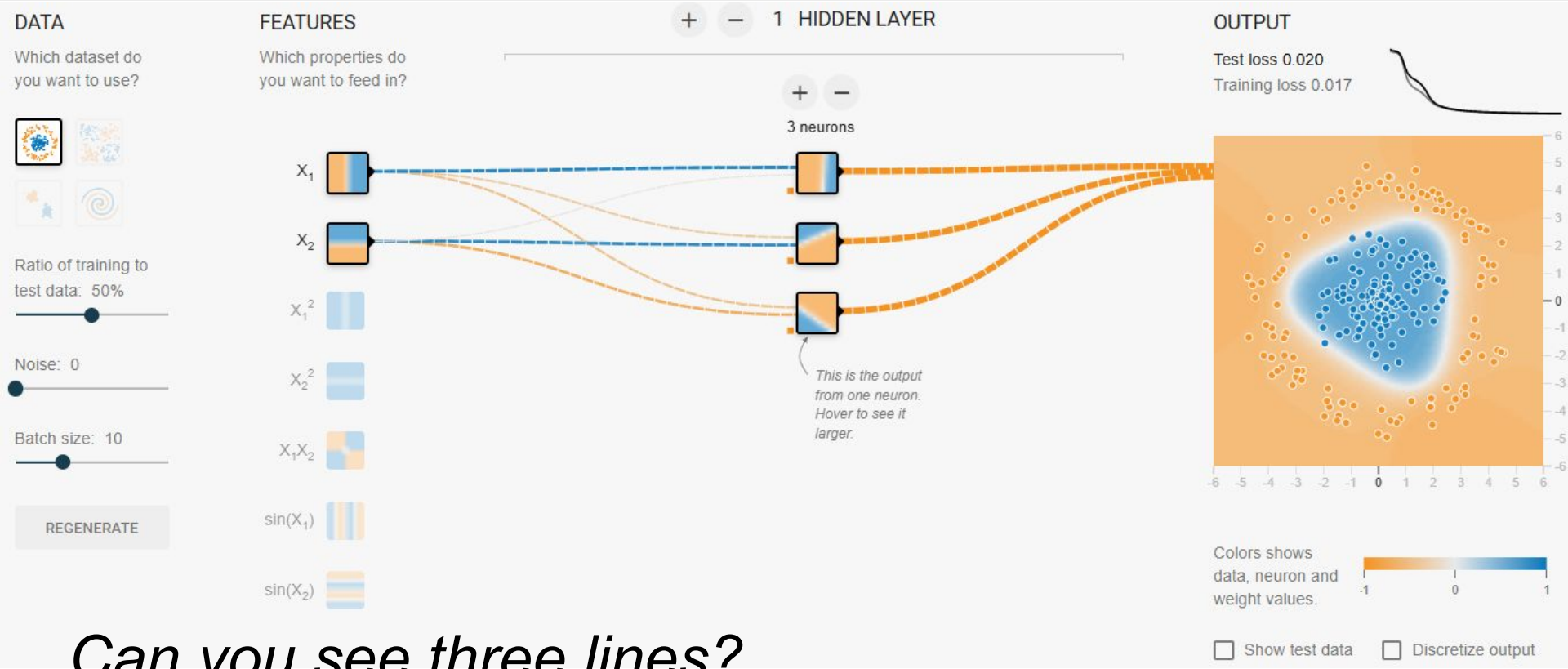
# Test #2: Non-linear clusters - one more layer



# Test #3: Donut - Can lines separate donut?

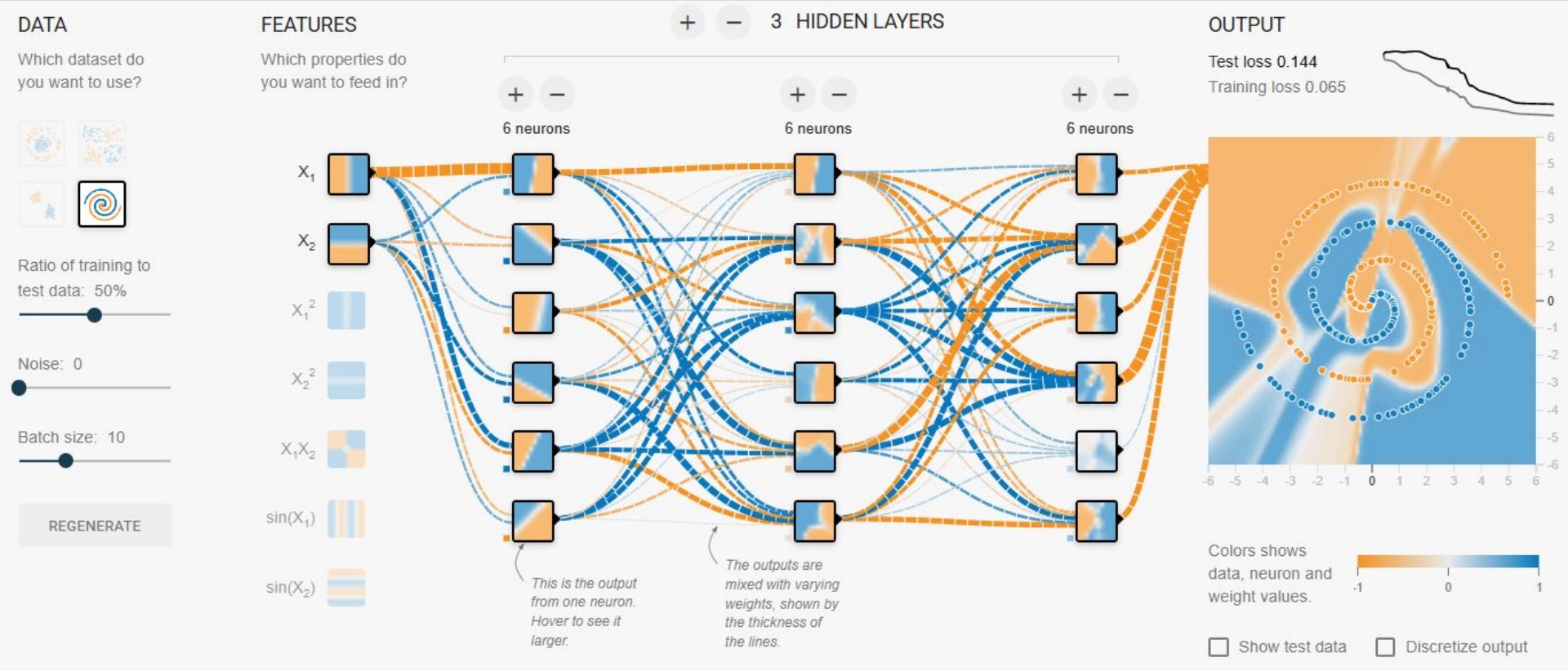


# Test #3: Donut - Can lines separate donut?

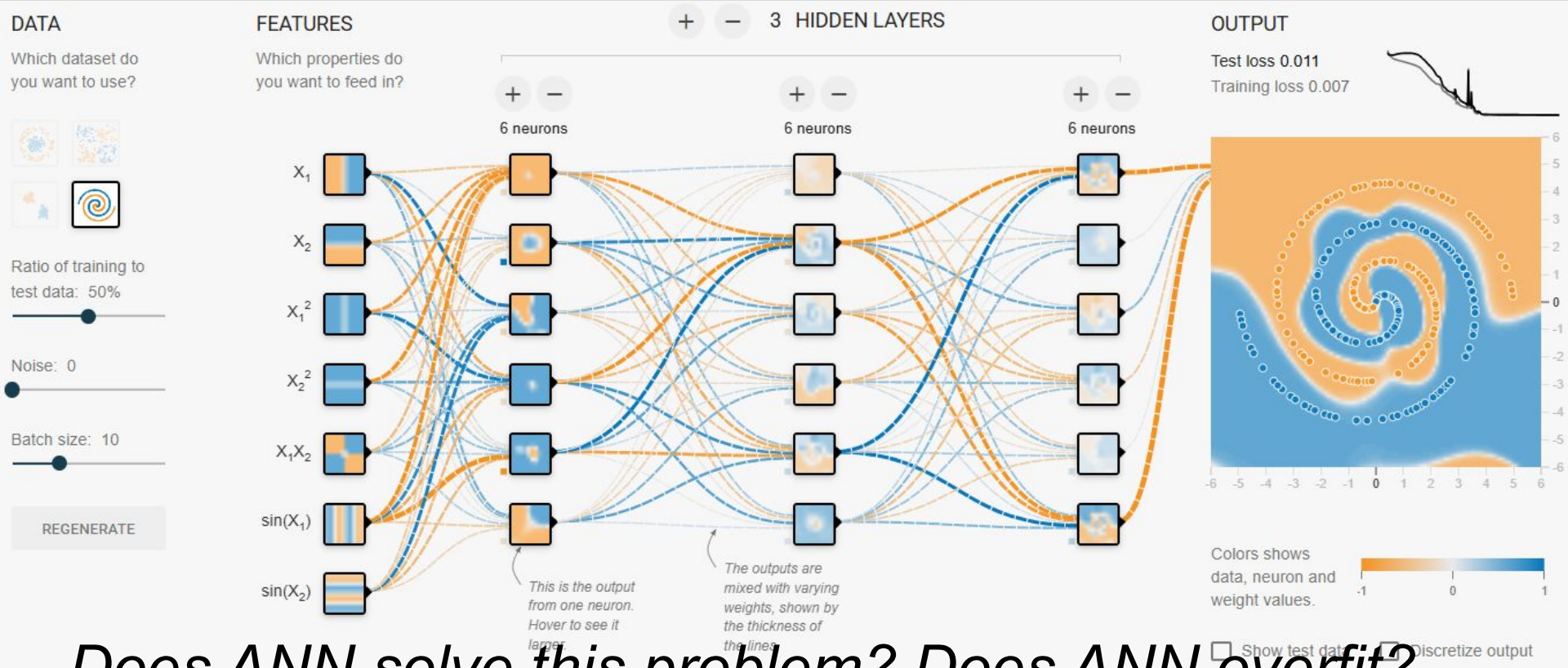



*Can you see three lines?*

# Test #4: Spiral cluster - nightmare example



# Test #4: Spiral cluster - nightmare example




*Does ANN solve this problem? Does ANN overfit?*

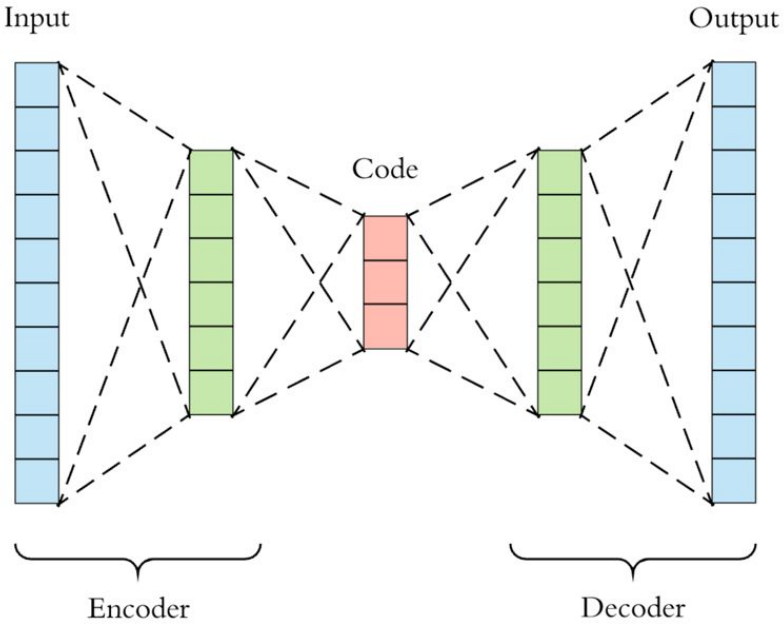
# Discussions on Artificial Neural Network

1. ANN can search for the decision boundaries, but ANN is prone to overfit the data easily.
2. ANN tries to learn every feature from the data, including outliers and extreme points. In other words, ANN can be perturbed by small input and errors.
3. ANN is a data-driven model / algorithm, relying heavily on the observations to make decisions, predictions or classifications.
4. ANN may have issues when the class is very imbalance.

# Autoencoder

# Introduction to Autoencoder

- Autoencoder is originally proposed for dimensionality reduction and feature learning (Code is a smaller representation)



# Introduction to Autoencoder

- In 1989 and 1991, Baldi and Hornik and Kramer proposed generalized PCA to autoencoders. At that time, the technique is named as **nonlinear PCA**.
- The first applications of autoencoder date to early 1990s, for dimensionality reduction or feature learning.
- In 2010s, the concept of autoencoder became widely used for learning generative models of data, e.g. **variational autoencoder (VAE)** in stable diffusion, **discrete VAE (dVAE)** in DALL-E 1.

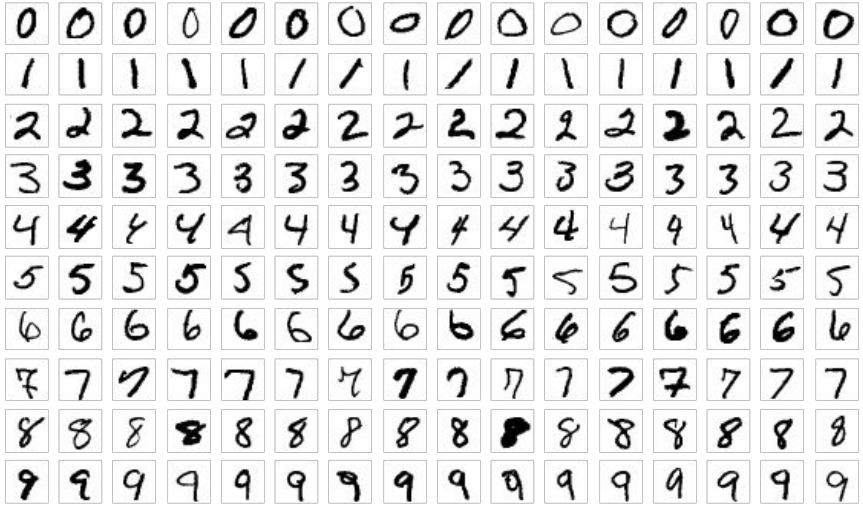
# Introduction to Autoencoder

- An autoencoder is a type of **unsupervised** artificial neural network used primarily for **dimensionality reduction** and **feature learning**. Its goal is to learn an efficient, compressed representation (encoding) of input data.
- It is called an **auto**-encoder because it uses the **same data** for both the **input** and the target **output**—it tries to reconstruct its own inputs.

1. **Encoder**: learns to reduce the input dimensions to a much smaller, dense representation called the bottleneck or latent space.
   - Input: High-dimensional data (e.g., a 784-pixel image)
   - Output: A compressed code (e.g., a vector of just 32 numbers)
2. **Bottleneck** (**Latent Space**): Its small size forces the network to learn the most important features and patterns of the data, discarding noise and irrelevant details.
3. **Decoder**: It tries to create an output as close as possible to the original input.
   - Input: A compressed code (e.g., a vector of just 32 numbers)
   - Output: A reconstruction of the original data (e.g., a 784-pixel image)

# Recall: Handwritten Digit Recognition

- Recognize handwritten digits using a machine learning approach (one of the first widely used ML systems)
- Let’s learn Autoencoder with MINST example.



Reference: http://yann.lecun.com/exdb/mnist/
http://yann.lecun.com/exdb/publis/pdf/matan-92.pdf

# Libraries and Datasets

```python
# conda install pytorch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 cpuonly -c pytorch
# reference 1:
# https://www.geeksforgeeks.org/deep-learning/implementing-an-autoencoder-in-pytorch/
# reference 2: https://github.com/SuchismitaSahu1993/Autoencoder-on-MNIST-in-Pytorch
import torch
from torch import nn, optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

tensor_transform = transforms.ToTensor()
dataset = datasets.MNIST(root="./data", train=True, download=True,
transform=tensor_transform)
loader = torch.utils.data.DataLoader(dataset=dataset, batch_size=32, shuffle=True)

```

*Download the public MINST dataset*

# The Autoencoder class

```python
class AE(nn.Module):
    def __init__(self):
        super(AE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(28 * 28, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, 36), nn.ReLU(),
            nn.Linear(36, 18), nn.ReLU(),
            nn.Linear(18, 9)
        )

        self.decoder = nn.Sequential(
            nn.Linear(9, 18),  nn.ReLU(),
            nn.Linear(18, 36), nn.ReLU(),
            nn.Linear(36, 64), nn.ReLU(),
            nn.Linear(64, 128), nn.ReLU(),
            nn.Linear(128, 28 * 28), nn.Sigmoid()
        )

```

*Encoder*

*Decoder*

# The Autoencoder class Forward Propagation

- Forward propagation is the process of taking an input data sample, passing it through the entire network layer by layer, and computing the output (the reconstruction) and the intermediate values (the encoded representation).
- If you wish to extract the **latent space**, the variable `encoded` stores the corresponding values.

```
    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded, encoded

```

# The Autoencoder Initialization and Training Loop

```
model = AE()
loss_function = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-8)
epochs = 20
outputs = []
losses = []
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

for epoch in range(epochs):
    for images, _ in loader:
        images = images.view(-1, 28 * 28).to(device)
        reconstructed = model(images)[0]
        loss = loss_function(reconstructed, images)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        losses.append(loss.item())

    outputs.append((epoch, images, reconstructed))
    print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.6f}")
```

*Use the MSE between the original image and reconstructed image as the error (loss)*

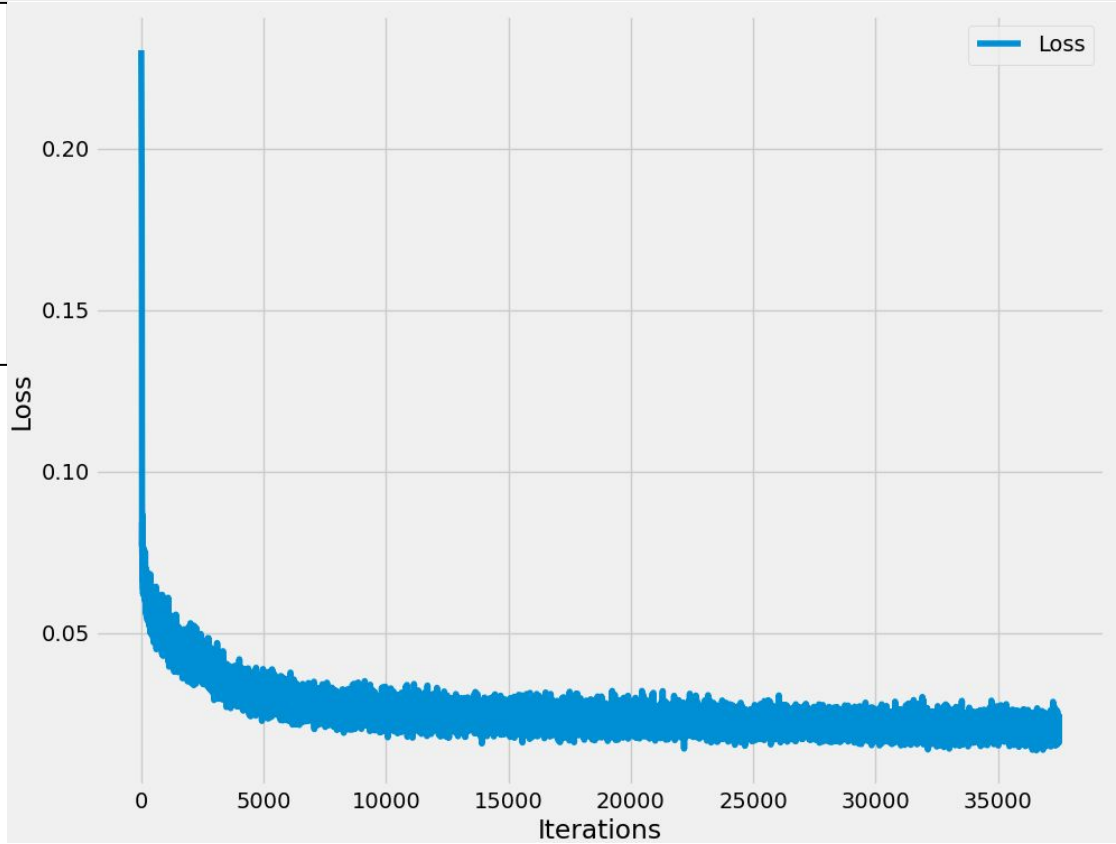
*Stack the error for visualization*

# The Autoencoder Training Loop

```
Epoch 1/20, Loss: 0.044368
Epoch 2/20, Loss: 0.037411
Epoch 3/20, Loss: 0.032105
Epoch 4/20, Loss: 0.031215
Epoch 5/20, Loss: 0.028185
Epoch 6/20, Loss: 0.027183
Epoch 7/20, Loss: 0.033509
Epoch 8/20, Loss: 0.028252
Epoch 9/20, Loss: 0.026210
Epoch 10/20, Loss: 0.022866
Epoch 11/20, Loss: 0.025566
Epoch 12/20, Loss: 0.022695
Epoch 13/20, Loss: 0.026058
Epoch 14/20, Loss: 0.022205
Epoch 15/20, Loss: 0.023880
Epoch 16/20, Loss: 0.023887
Epoch 17/20, Loss: 0.023685
Epoch 18/20, Loss: 0.020830
Epoch 19/20, Loss: 0.018869
Epoch 20/20, Loss: 0.022205
```

# The Autoencoder Visualizing the Training

```
plt.style.use('fivethirtyeight')
plt.figure(figsize=(8, 5))
plt.plot(losses, label='Loss')
plt.xlabel('Iterations')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



# Data Representation by Autoencoder

# More about the Latent Space

- The latent space, also known as the coding space or bottleneck, describes the low-dimensional representation of input data, **a compressed and abstract representation of the data** that the encoder aims to optimize for and the decoder then aims to expand back to the input form.
- In terms of the mathematical modeling, we can understand the latent space in the context of autoencoders as a mapping: **Encoder: $X \rightarrow Z$**
- Where X represents the input data and Z represents the latent space, or the codings that represent the input data. And in the encoder space, we'are aiming to perform **dimensionality reduction**.
- And the reverse can be said for the decoder: **Decoder: $Z \rightarrow \ddot{X}$**
- Here, the mapping is from the latent space, Z, to a reconstructed form of the input data, $\ddot{X}$. And in the decoder space, we're aiming to perform **dimensionality expansion**.

# Common Use Cases of Autoencoder

1. **Data Compression**: By reducing the dimensionality of data, autoencoders efficiently compress information.
2. **Data Denoising**: They can distinguish between noise and signal, useful in processing noisy datasets.
3. **Anomaly Detection**: Autoencoders can identify deviations from normal or expected patterns, making them effective in fraud detection, medical diagnostics, and quality control.
4. **Data Visualization**: By projecting high-dimensional data onto lower dimensions, such as 2D or 3D, autoencoders assist in data visualization tasks.
5. **Feature Engineering**: Autoencoders are used to learn efficient feature representations, particularly when labeled data is scarce or unavailable.
6. **Data Generation**: Certain autoencoder variants, such as VAEs, are capable of generating new data samples from the learned latent space, making them useful in generative tasks like image and text generation.

# Common Autoencoder Limitations

1. **Insufficient training data**: Autoencoders are unsupervised models learning from its own data. In real problems, there are always exceptions and new examples.
2. **Training the wrong use case**: There are always errors in real data.
3. **Too lossy**: The compression degradation in the encoder always drops lots of information in the latent space.
4. **Narrow bottleneck layer**: How can we determine the right size of the bottleneck layer?
   - For MINST problem, the answer is trivial (size of encoded = 9 or 10)
   - For classification problems, the answer sounds trivial. How about regression problems, clustering problems, and other real problems?
5. **Imperfect decoding**: The decode (decompression) process can never be perfect.

# How to determine the size of latent space?

1. **Knowledge from the problem domain**, e.g. MINST dataset problem.
2. **Progressive Reduction**: An empirical and iterative approach
   - Start with a guess
   - Train the autoencoder
   - Evaluate the results
   - Iterate the process by increasing or decreasing the guess
3. **Intrinsic Dimensionality** of the data
   - Analyze explained variance ratio based on the PCA analysis to estimate the size of the latent space
4. **Rule of Thumb**, e.g. design the encoder through downsampling the input progressively, and use $2^n$ naturally for images

- Remark: latent space can be a 2D tensor, instead of a 1D vector.

# Google Colaboratory

Link: https://colab.google/notebooks/

# Google Colaboratory

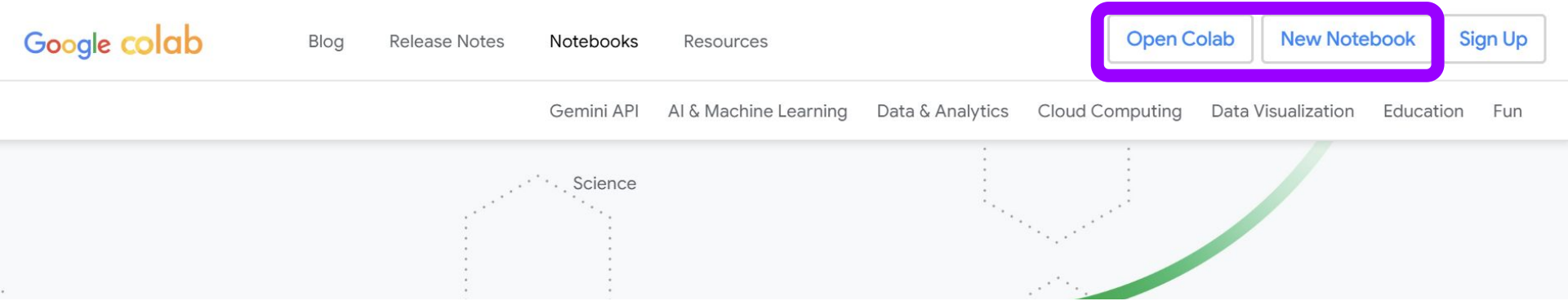
- Google Colaboratory, or Google Colab for short, is a free, cloud-based Jupyter Notebook environment provided by Google. It allows users to write and execute Python code through the browser, especially suited for machine learning, data analysis, and education.
- Link: https://colab.google/notebooks/

# Google Colaboratory Features

1. **Jupyter Notebook Interface**: It uses a Jupyter notebook-style interface, enabling users to create documents that contain live code, equations, visualizations, and narrative text.
2. **Free Access to GPUs**: Users can access powerful hardware accelerators like GPUs and TPUs, which are especially useful for machine learning and data analysis tasks.
3. **Integration with Google Drive**: You can save your notebooks directly to Google Drive, making it easy to share and collaborate with others.
4. **Libraries and Frameworks**: Colab comes pre-installed with many popular Python libraries (like TensorFlow, PyTorch, Pandas, and NumPy) used for data science and machine learning.
5. **Collaboration**: Multiple users can work on the same notebook simultaneously, similar to Google Docs.

# Working with a Google Colab Notebook

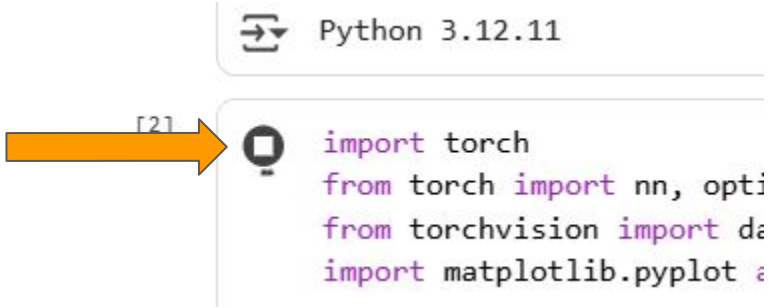
1. **Create a New Notebook**: Click on "File" in the menu and select "New Notebook." Alternatively, you can open an existing notebook from Google Drive or GitHub.



# Working with a Google Colab Notebook

2. **Familiarize Yourself with the Interface**: The interface consists of code cells and text cells. You can run code in the cells using the play button or by pressing **Shift + Enter**.

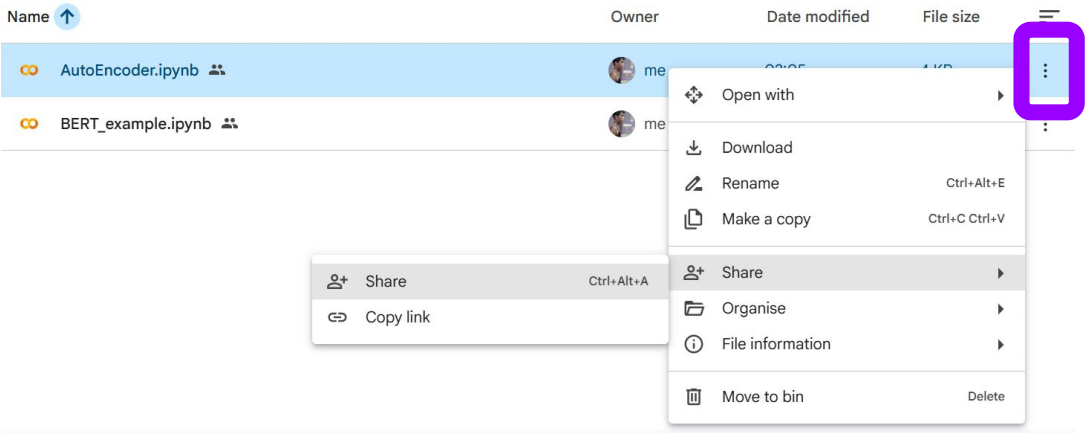
After pressing **Shift + Enter**

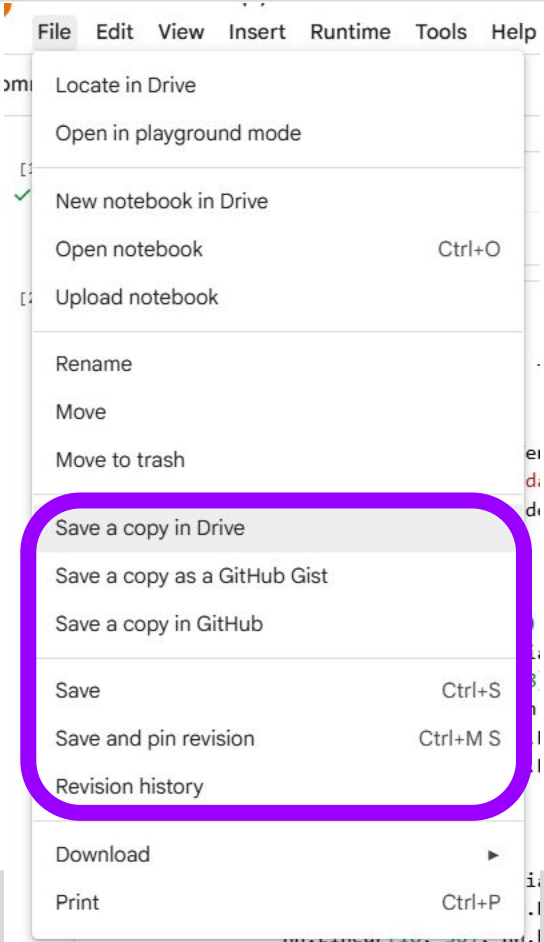



3. **Write and Execute Code**: Start writing Python code in the code cells. You can import libraries, load datasets, and perform computations.

# Working with a Google Colab Notebook

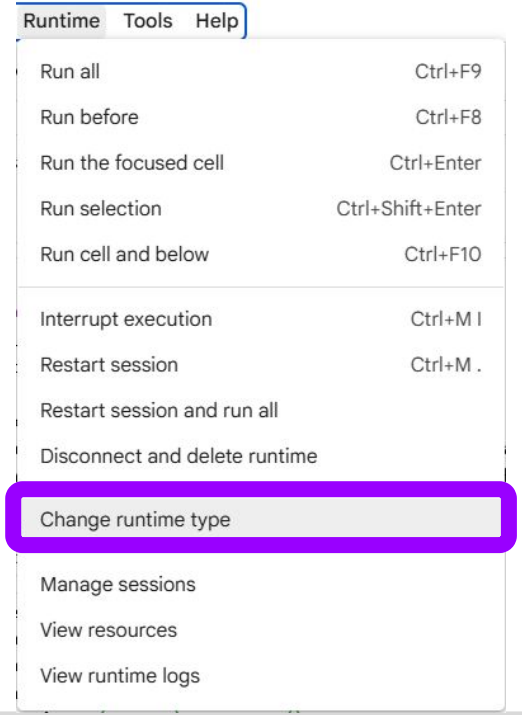
4. **Save Your Work**: Click on "File" > "Save a copy in Drive" to save your notebook to Google Drive for future access.
5. **Share Your Notebook**: To collaborate, click on the "Share" button in the upper right corner and invite others by entering their email addresses.

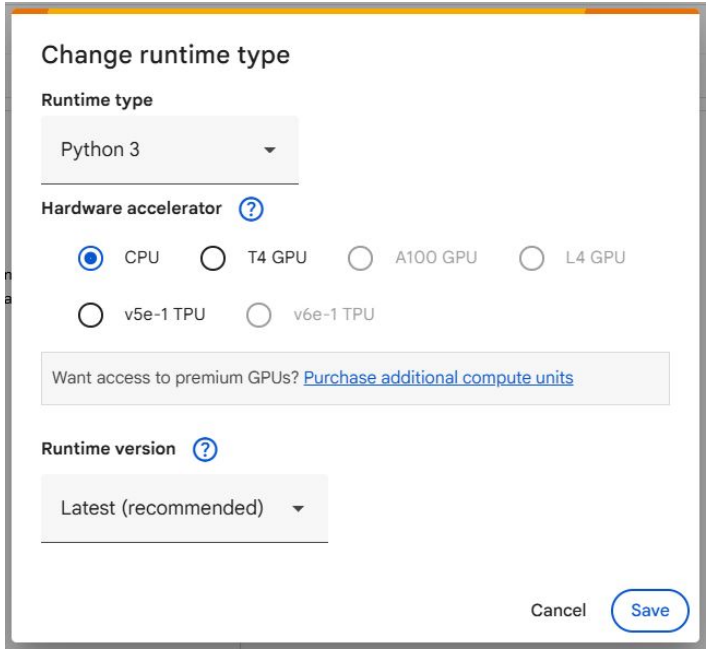


# Working with a Google Colab Notebook

6. **Utilize Hardware Acceleration**: If you're working on heavy computations, go to "Runtime" > "Change runtime type" and choose GPU or TPU as needed.



# Example: Autoencoder

AutoEncoder.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[ ]
```python
!python --version
```

```
Python 3.12.11
```

[ ]
```python
import torch
from torch import nn, optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

tensor_transform = transforms.ToTensor()
dataset = datasets.MNIST(root="./data", train=True, download=True, transform=tensor_transform)
loader = torch.utils.data.DataLoader(dataset=dataset, batch_size=32, shuffle=True)

class AE(nn.Module):
    def __init__(self):
        super(AE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(28 * 28, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, 36), nn.ReLU(),
            nn.Linear(36, 18), nn.ReLU(),
            nn.Linear(18, 9)
        )
```

Link: https://colab.research.google.com/drive/1kQuYRpzclZe76kfKaLXUUobQI6ECXhh8?usp=sharing

# Example: Transformer

BERT_example.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] 1m

```python
# Install the Hugging Face Transformers library
!pip install transformers

# Import the necessary libraries
import torch
from transformers import BertTokenizer, BertForSequenceClassification

# Load the pre-trained BERT model and tokenizer
model_name = 'nlptown/bert-base-multilingual-uncased-sentiment'
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForSequenceClassification.from_pretrained(model_name)

# Function to classify sentiment
def classify_sentiment(text):
    inputs = tokenizer(text, return_tensors='pt', truncation=True, padding=True)
    with torch.no_grad():
        outputs = model(**inputs)
    logits = outputs.logits
    predicted_class = torch.argmax(logits, dim=1).item()
    return predicted_class

# Example usage
text = "I love using Google Colab for my projects!"
predicted_sentiment = classify_sentiment(text)
print(f"Predicted Sentiment Class: {predicted_sentiment}")
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.56.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.19.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.34.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.0)
```

Link: https://colab.research.google.com/drive/1qJt9wJzpbqO1Dvtvyb65kklkk0dvHO7L?usp=sharing

# Transformer

# From Autoencoder to Transformer

- Autoencoder learns an efficient compressed representation (encoding) of the input data and then learns to reconstruct the original input from this compression. **It is all about the data itself.**
  - Common applications include data compression, data denoising, detection of anomalous data points, …. etc.
- Transformer understands the context of every element in a sequence by relating it to all other elements, and then uses this rich contextual understanding to perform a task. **It is about relationships and context.**
  - Common applications include language translation, question answering, text generation, text summarization, … etc

# Introduction to Transformer

- A transformer is a type of deep learning architecture, introduced in the 2017 paper "Attention Is All You Need" by Google researchers. It is designed primarily for handling sequential data (i.e. natural language).
- Its core innovation is the self-attention mechanism, which allows it to weigh the importance of all parts of the input data when making a prediction, without relying on the sequential order of processing.

# From Autoencoder to Transformer

| Features | Autoencoder | Transformer |
|---|---|---|
| Primary purpose | Unsupervised learning | Sequence model |
| Core mechanism | Bottleneck layer | Self-attention mechanism |
| Input | Static data, e.g. images, sparse matrix, … etc | Sequential data, e.g. natural language, time-series data |
| Output | A reconstructed version of the input | A transformed or generated sequence, e.g. translated sentence, summary |
| Paradigm | Compression and reconstruction | Contextual representation and transformation |
| Philosophy | It is all about data itself | It is all about relationships and context |

# Transformer Model

- This architecture aims to solve sequence-to-sequence tasks while handling long-range dependencies with ease. It relies entirely on **self-attention** to compute representations of its input and output without using sequence-aligned RNNs or convolution.
- Self-attention is a rich representation which captures the interdependencies between the words compared to single word embeddings. Self-attention adheres to long-term dependencies.
- Three types of attention:

1. Within encoder
2. Encoder - decoder
3. Within decoder

# Attention Mechanism

- Attention mechanisms is a layer of neural networks added to deep learning models to focus their attention to specific parts of data, based on different weights assigned to different parts.

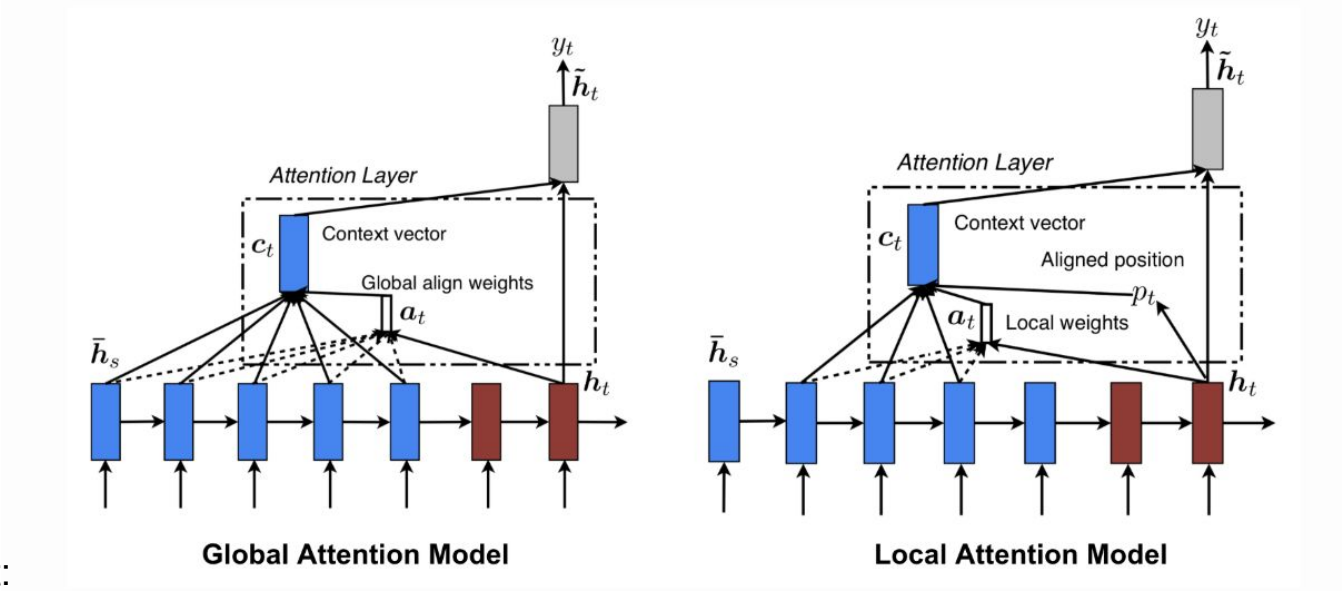
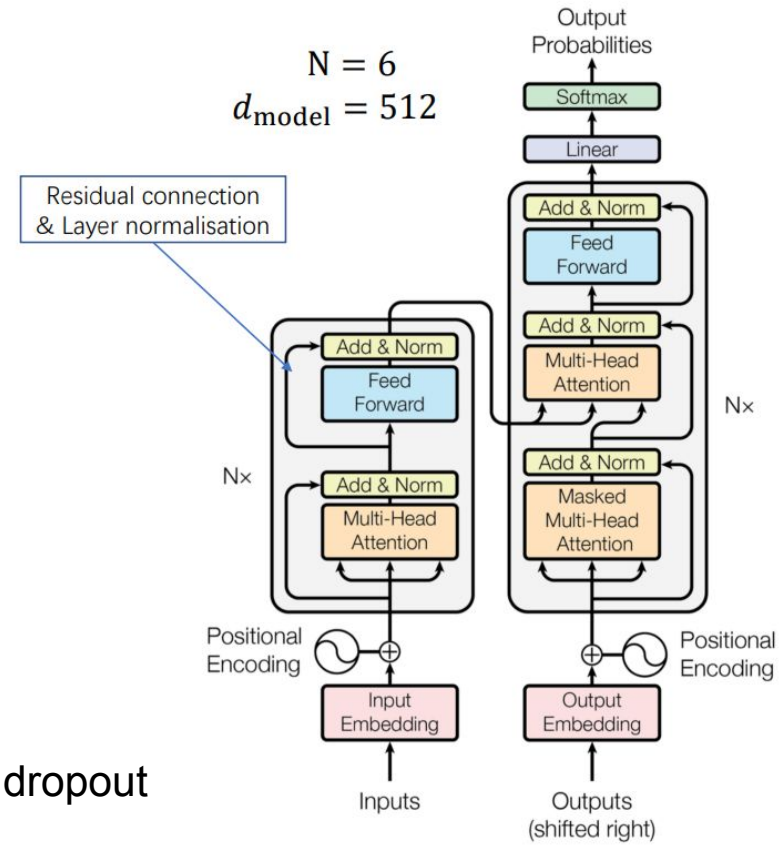



Image credit:
https://www.analyticsvidhya.com/blog/2019/11/comprehensive-guide-attention-mechanism-deep-learning/

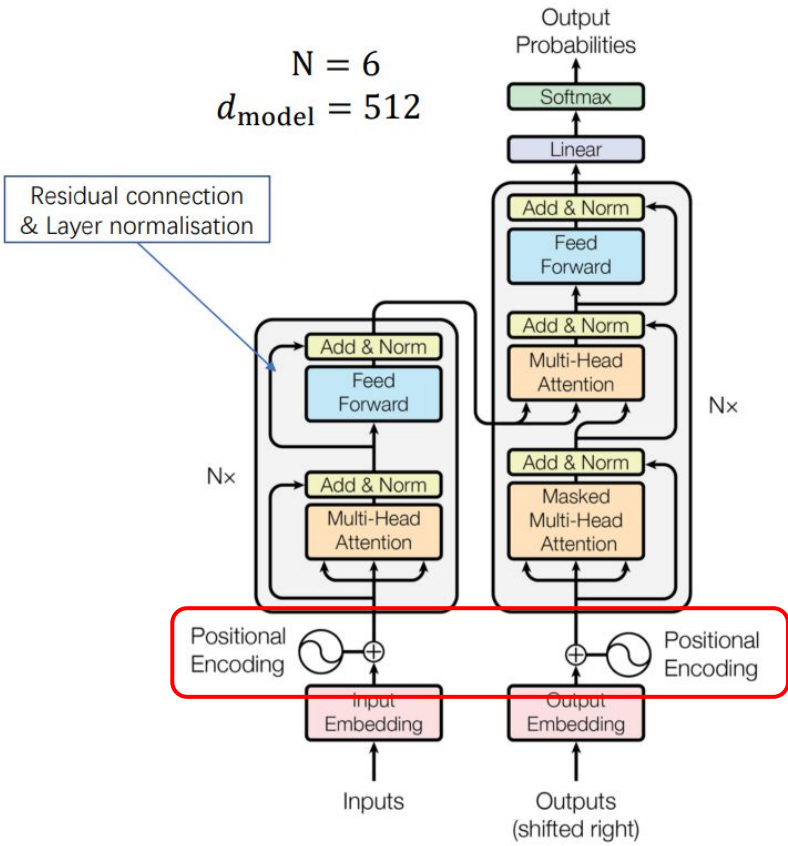
# Transformer Model

- Encoder (Left)
  - Stack of 6 layers
  - Self-attention + feed forward network (FFN)
- Decoder (Right)
  - Stack of 6 layers
  - Self-attention, Source-target attention + FFN
- Components
  - Positional encoding
  - Multi-head attention
  - Position-wise FFN
  - Residual connection
- Regularization
  - Residual dropout, label smoothing, attention dropout



# Positional Encoding

- Add information about the position of the words in the sentence



# Transform's Self-Attention Mechanism (Simplified)

- Output and input have the same sequence length and dimension. Weight each value by similarity of the corresponding query and key.
- For each sequence position output sum up the weighted values.

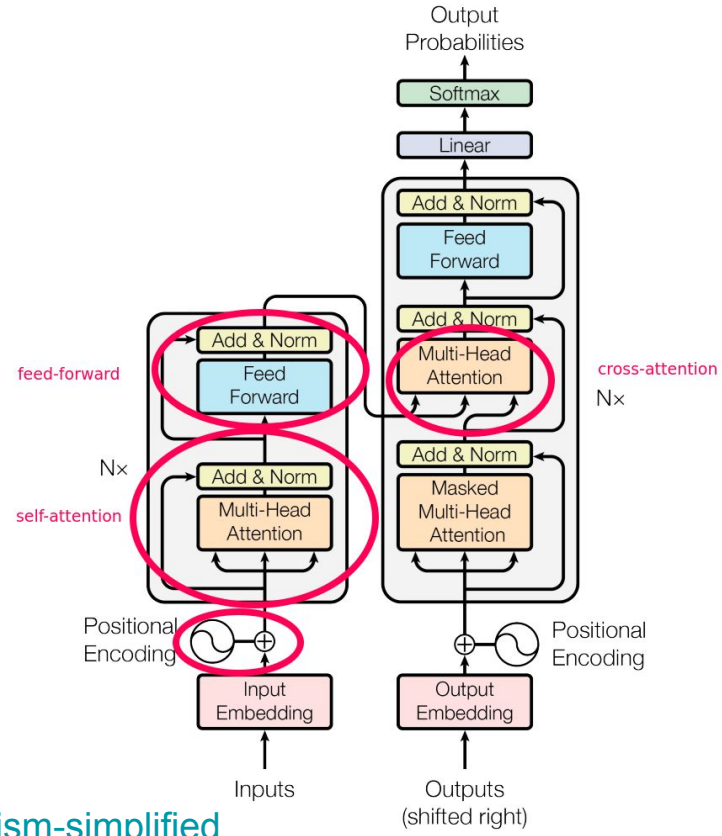



Image credit:
[https://vaclavkosar.com/ml/transformers-self-attention-mechanism-simplified](https://vaclavkosar.com/ml/transformers-self-attention-mechanism-simplified)

# Transform's Cross-Attention Mechanism (Simplified)

- Except for inputs, cross-attention calculation is the same as self-attention.
- Cross-attention combines asymmetrically two separate embedding sequences of same dimension, in contrast self-attention input is a single embedding sequence.
- One of the sequences serves as a query input, while the other as a key and value inputs. Alternative cross-attention in SelfDoc, uses query and value from one sequence, and key from the other.

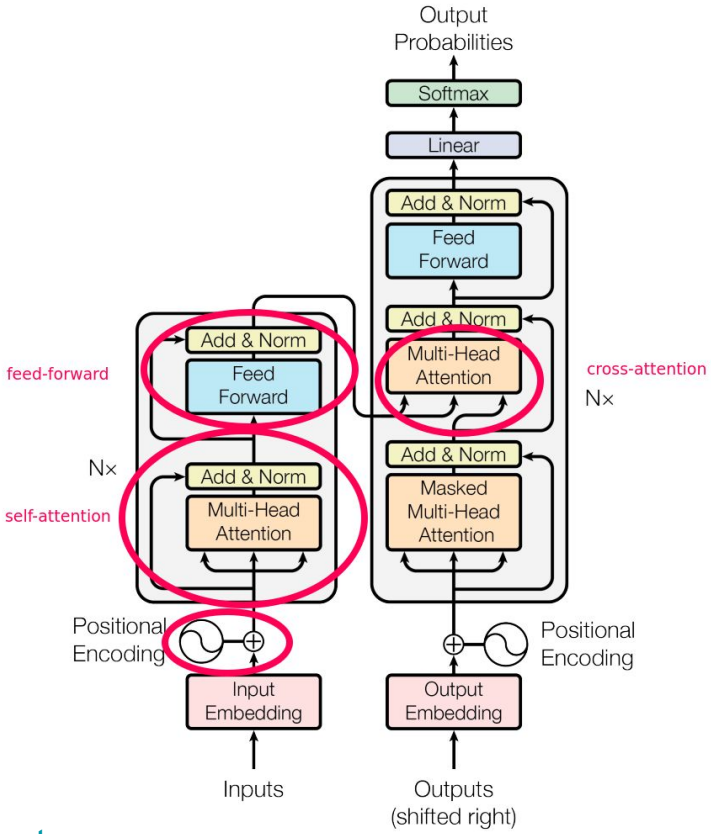
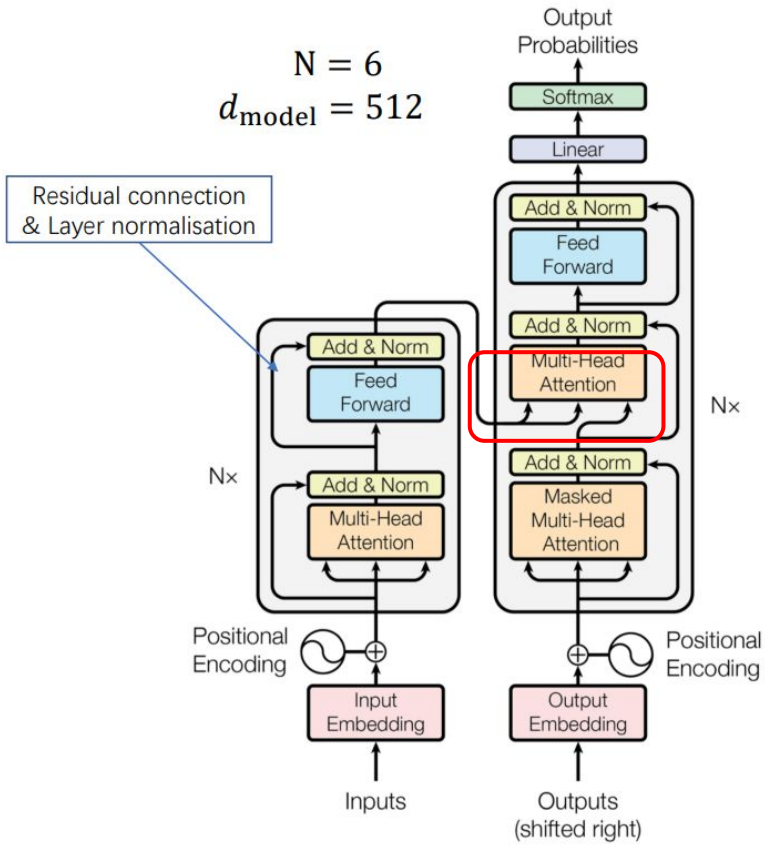



Image credit:
https://vaclavkosar.com/ml/cross-attention-in-transformer-architecture

# Source-Target or Self Attention

- Source-Target Attention
  - Used in the second multi-head attention layer of the transformer decoder layer
- Self-attention
  - Used in the multi-head attention layer of the transformer econder layer and the first one of the transformer decoder layer
- What's the different?
  - Query from encoder → S-T attention
  - Query from decoder → self attention



# Build Transformer Model - Class

```python
class Transformer(nn.Module):
    def __init__(self, transformer, output_dim, freeze):
        super().__init__()
        self.transformer = transformer
        hidden_dim = transformer.config.hidden_size
        self.fc = nn.Linear(hidden_dim, output_dim)
        if freeze:
            for param in self.transformer.parameters():
                param.requires_grad = False

    def forward(self, ids):
        # ids = [batch size, seq len]
        output = self.transformer(ids, output_attentions=True)
        hidden = output.last_hidden_state
        # hidden = [batch size, seq len, hidden dim]
        attention = output.attentions[-1]
        # attention = [batch size, n heads, seq len, seq len]
        cls_hidden = hidden[:, 0, :]
        prediction = self.fc(torch.tanh(cls_hidden))
        # prediction = [batch size, output dim]
        return prediction
```

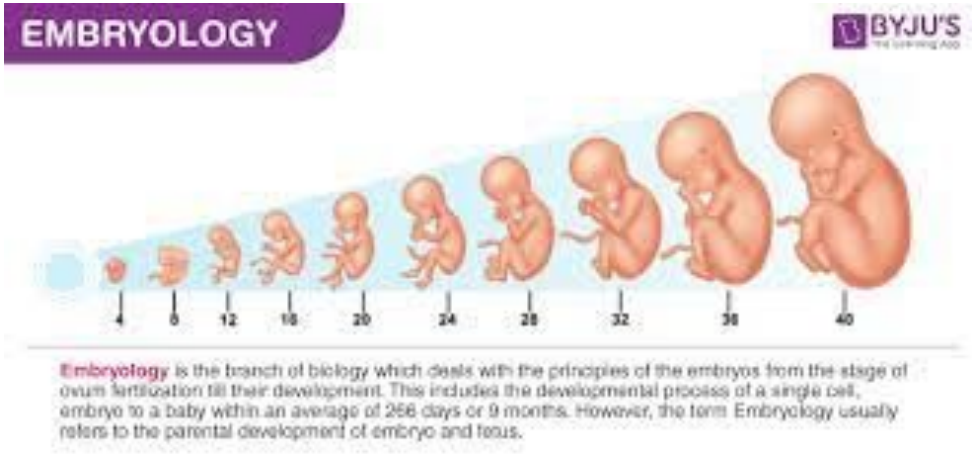
*It's just a reference, no one train a transformer from scratch now.*

Reference:
https://github.com/bentrevett/pytorch-sentiment-analysis/blob/main/4%20-%20Transformers.ipynb

# Always Use Pre-trained Model for Transformer

```
transformer_name = "bert-base-uncased"
tokenizer = transformers.AutoTokenizer.from_pretrained(transformer_name)
transformer = transformers.AutoModel.from_pretrained(transformer_name)
```

- It is very expensive to train a Transformer model from scratch.
- Use **transformer embryos** (**pre-trained models** with its **tokenizer**).
- Then, re-train the embryos with your data.
- According to one of the FYP student (Edwin Cheung)
  - Training a BERT model with 3 billions words took **8 days with TPU v3**.
  - Re-training a BERT model with 3 billions words took **2-3 hours with TPU v3**.



# Data Representation by Transformer

# Steps on using Transformer

1. Import libraries
2. Prepare the data
   - **Tokenization** and **vocabulary creation** with **special tokens**
   - Use **masks** to prevent the decoder from peeking, as there are always unseen tokens and new phases
3. Define model with a pre-trained embryo
4. Fine-tune the model with your data
5. Generate output

# Input Tokens: From Words to Indices

- The input sentence ("The cat sat on the mat") is split into smaller units called tokens (e.g., words, subwords: `["The", "cat", "sat", "on", "the", "mat"]`).
- Representation: Each token is mapped to a unique integer ID using a vocabulary dictionary. E.g.
  - `"The" -> 101`
  - `"cat" -> 202`
  - `...`
  - `"mat" -> 999`
- Shape: A sequence of integers. For a batch of sentences, this is a 2D tensor of shape (batch_size, sequence_length). E.g.
  - `[101, 202, 303, 404, 101, 999]`

# Token Embeddings: Capturing Meaning

- An embedding layer (a simple lookup table) is used. Each token ID is used as an index to retrieve its corresponding vector.
- Representation: Each token is now a vector of size d_model (e.g., 512, 768). The values in this vector are learned during training. Tokens with similar meanings will have similar vector values (e.g., "cat" and "dog" will be closer in this space than "cat" and "airplane").
- Shape: The 2D integer tensor becomes a 3D float tensor: (batch_size, sequence_length, d_model).
  - 
```
[ [0.1, 0.2, 0.3, ...],  # Vector for "The" (ID 101)
  [1.1, 1.2, 1.3, ...],  # Vector for "cat" (ID 202)
  [2.1, 2.2, 2.3, ...],  # Vector for "sat" (ID 303)
  ... ]
```

# Positional Encodings: Adding the Concept of Order

- A fixed or learned pattern is added to the token embeddings. This pattern uniquely identifies each position in the sequence.
- Learned Positional Embeddings: A separate embedding layer that learns a unique vector for each position (e.g., position 0 gets a vector, position 1 gets another). This is simpler but fixed to a maximum sequence length seen during training.
- Shape: The positional encoding tensor has the same shape (batch_size, sequence_length, d_model). It is element-wise added to the token embeddings.

# Positional Encodings: Adding the Concept of Order

- Example:

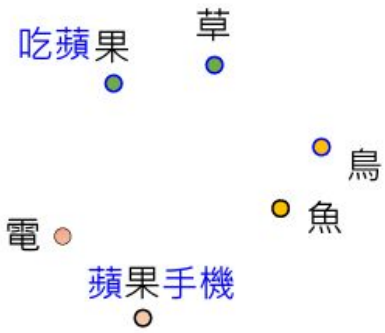
```
Token Embedding:    [ [0.1, 0.2, ...],  # "The"
                      [1.1, 1.2, ...],  # "cat"
                      [2.1, 2.2, ...] ] # "sat"
+
Positional Encoding:[ [0.0, 0.1, ...],  # Position 0
                      [1.0, 1.1, ...],  # Position 1
                      [2.0, 2.1, ...] ] # Position 2
=
Input Representation:
                  [ [0.1, 0.3, ...],  # "The" at pos 0
                    [2.1, 2.3, ...],  # "cat" at pos 1
                    [4.1, 4.3, ...] ] # "sat" at pos 2
```

# Contextualized Embeddings: The Output of Self-Attention

- As the data passes through each encoder/decoder block, the self-attention mechanism allows each token to "look" at every other token in the sequence. It computes a weighted average of the representations of all other tokens, updating its own representation.
- Representation: The output for each token is now a vector that is a blend of itself and the tokens it found to be most relevant. The representation for "bank" will be different if it's attended to "river" vs. "account".
- Shape: The shape remains (batch_size, sequence_length, d_model), but the values inside the vectors are now rich with context.

# Visualizing Contextualized Embeddings

- The tokens with similar meaning have similar embedding.
- Context of the token is considered in the embedding.
- Cosine similarity is computed.



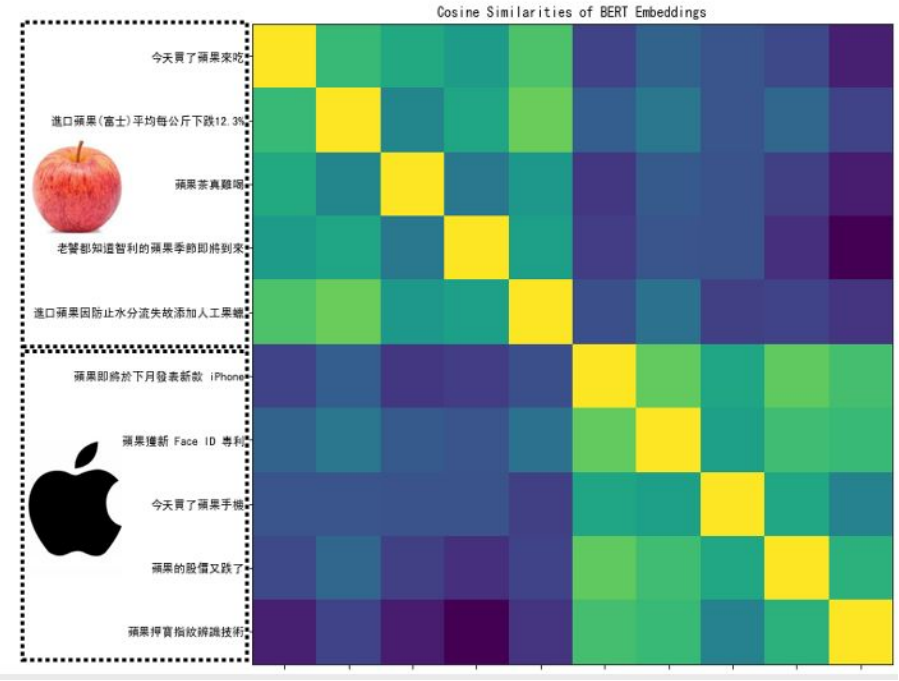




Image credit:
[https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/bert_v8.pdf](https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/bert_v8.pdf)

# Example: Retrieve Embeddings from BERT

```python
import torch
from transformers import BertTokenizer, BertModel

# Load pre-trained model and tokenizer
model_name = 'bert-base-uncased'
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertModel.from_pretrained(model_name)

# Set the model to evaluation mode
model.eval()

# Example input text
text = "Hello, how are you?"

# Tokenize input text
inputs = tokenizer(text, return_tensors='pt')

```

*Model and its tokenizer must work together*

Link: https://colab.research.google.com/drive/1uGKyBYS8_JHXlvcFSJxZuWpAhjdHwKI0?usp=sharing

# Example: Retrieve Embeddings from BERT

```python
# Extract the last hidden state
last_hidden_state = outputs.last_hidden_state

# Now `last_hidden_state` contains the latent representation
print(last_hidden_state)
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 2.17kB/s]
vocab.txt: 100%              232k/232k [00:00<00:00, 5.07MB/s]
tokenizer.json: 100%         466k/466k [00:00<00:00, 8.55MB/s]
config.json: 100%             570/570  [00:00<00:00, 33.5kB/s]
model.safetensors: 100%      440M/440M [00:06<00:00, 75.0MB/s]
tensor([[[-0.0824,  0.0667, -0.2880,  ..., -0.3566,  0.1960,  0.5381],
         [ 0.0310, -0.1448,  0.0952,  ..., -0.1560,  1.0151,  0.0947],
         [-0.8935,  0.3240,  0.4184,  ..., -0.5498,  0.2853,  0.1149],
         ...,
         [-0.2812, -0.8531,  0.6912,  ..., -0.5051,  0.4716, -0.6854],
         [-0.4429, -0.7820, -0.8055,  ...,  0.1949,  0.1081,  0.0130],
         [ 0.5570, -0.1080, -0.2412,  ...,  0.2817, -0.3996, -0.1882]]])
```

# Limitations of Transformers

1. Very expensive computational resources
2. Requiring an accurate or relevant pre-trained model
3. Restrictions on the length of the sequence
4. Limited interpolations on the contextualized embeddings
5. Long time for generating response or output

# What’s Next?

- We have learnt how to simplify the representation of all kinds of data using deep neural network
  - **Autoencoder** and **Transformer**
- We are going to learn **different machine learning algorithms (data models)** for solving different data science problems
  - Supervised learning, Unsupervised learning, Semi-supervised learning
  - Clustering, Classification, Regression, Reinforcement learning